



Helm ve Kustomize

Kubernetes ve Konteyner Dünyası



Bu Konuda

- Kustomize: base/overlay modeli, ortam ayrımı, generator'lar ve patch'ler
- Helm: chart, release ve repository; dizin yapısı, template ve values
- Release yaşam döngüsü: `install`, `upgrade`, `rollback`, `uninstall`; `--atomic` ve `--wait`
- Yerel doğrulama: `helm template`, `helm lint`, `kubectl kustomize`

Kustomize





Kustomize Nedir?

- Aynı uygulamanın manifesti ortama göre değişir: imaj etiketi, replica sayısı, ConfigMap verisi
- Kustomize bu farkı manifest kopyalamadan yönetir: orijinal YAML'lar girdi olarak kalır, özelleştirme `kustomization.yaml` dosyasında tarif edilir^[1]
- Üç işlev grubu: kaynak üretmek (ConfigMap/Secret), kaynaklara ortak alanlar eklemek, kaynakları birleştirip özelleştirmek^[1]
- kubectl 1.14'ten beri kustomization dosyasıyla yönetimi destekler; `kubectl kustomize` render eder, `kubectl apply -k` uygular^[1]
- Declarative apply akışına dokunmaz: manifestler kaynak kontrolünde durur, `kubectl apply` farkı hesaplayıp birleştirir; Kustomize uygulanacak manifestleri üreten ön katmandır^[2]

```
kubectl kustomize <dizin>    # özelleştirilmiş YAML'ı yazdırır
kubectl apply -k <dizin>     # render edip uygular
```



Base ve Overlay: Ortam Ayrımı

- **Base:** kendi `kustomization.yaml` 'ı olan, kaynak kümesini ve ortak özelleştirmesini taşıyan dizindir; overlay'lardan habersizdir ve birden çok overlay'da kullanılabilir^[1]
- **Overlay:** `resources` içinde başka kustomization dizinlerini gösteren dizindir; bir veya daha çok base'i birleştirip üstüne kendi farklarını ekler^[1]

```
base/  
  kustomization.yaml      # resources: [deployment.yaml, service.yaml]  
  deployment.yaml  
  service.yaml  
overlays/  
  dev/kustomization.yaml  # resources: [../../base] + namePrefix: dev-  
  staging/kustomization.yaml # resources: [../../base]  
  prod/kustomization.yaml  # resources: [../../base] + namePrefix: prod-
```

- Dev/staging/prod ayrımı böyle kurulur: üç ortam aynı base'i tüketir, farkı yalnız kendi dizininde taşır; base'i değiştirmeden yeni ortam eklenir^[1]



resources ve İmaj Değiştirme

- `resources` : kustomization'a dahil edilecek dosyaların listesi; Deployment + Service gibi kaynaklar tek kustomization'da birleştirilir^[1]
- `images` : patch yazmadan imaj değiştirir; `name` ile eşleşen imaja `newName` ve `newTag` uygulanır^[1]

```
resources:  
- deployment.yaml  
- service.yaml  
images:  
- name: nginx  
  newName: kayit.ornek/nginx  
  newTag: "1.27.1"
```

- Render sonucunda `image: nginx` → `image: kayit.ornek/nginx:1.27.1` ^[1]
- İmaj değişikliği ortam overlay'ına konursa base tek imaj adıyla kalır; her ortam kendi tag'ini seçer^[1]



namePrefix, nameSuffix ve Ortak Etiketler

- Ortak alanlar ("cross-cutting fields") tüm kaynaklara uygulanır: `namespace`, `namePrefix`, `nameSuffix`, `commonAnnotations` ve etiketler^[1]
- `labels` etiketleri her kaynağın metadata'sına ekler; `includeSelectors: true` ile selector ve Pod şablonuna da işlenir^[1]

```
namespace: uygulama
namePrefix: dev-
nameSuffix: "-v2"
labels:
- pairs:
    takım: platform
  includeSelectors: true
```

- Sonuç: `dev-web-v2` adı, `namespace: uygulama` ve her katmanda `takım: platform` etiketi^[1]



configMapGenerator ve secretGenerator

- `configMapGenerator` üç girdi alır: `files` (dosya içeriği tek anahtar), `envs` (`.env` satırları ayrı anahtar), `literals` (anahtar=değer)^[1]
- `secretGenerator` aynı girdilerle Secret üretir; değerler base64 kodlanır, tür `Opaque` olur^[1]
- Üretilen isme içerik hash'i eklenir; bu, içerik değiştiğinde yeni bir kaynak üretilmesini sağlar^[1]
- Deployment'ta generator adıyla referans verilir; Kustomize adı üretilen adla otomatik değiştirir^[1]
- `generatorOptions.disableNameSuffixHash: true` hash sonekini kapatır^[1]

```
configMapGenerator:  
- name: app-config  
  literals:  
    - MESAJ=merhaba  
secretGenerator:  
- name: app-secret  
  literals:  
    - PAROLA=gizli
```




Strategic Merge ve JSON Patch

- `patches` alanı sırasıyla uygulanan yama listesidir; hedef `group`, `version`, `kind`, `name`, `namespace`, `labelSelector`, `annotationSelector` ile seçilir^[1]
- Strategic merge: yama, hedef kaynağın türünü bilerek birleştirilir; ama her kaynak ve alan strategic merge desteklemez^[1]
- JSON patch (RFC 6902): desteklenmeyen alanlar için; `target` alanı zorunludur, `op` / `path` çiftleriyle çalışır^[1]

```
patches:
- target:
  kind: Deployment
  name: my-nginx
  patch: |-
    - op: replace
      path: /spec/replicas
      value: 3
```



kubectl kustomize ve kubectl apply -k

- `kubectl kustomize DIR` : yerel render; `DIR` verilmezse `.` varsayılır, git deposu URL'i de kabul edilir; cluster'a bağlanmaz^[3]
- `kubectl apply -k DIR` : render edip uygular; `-k` kustomization dizini bekler, `-f / -R` ile birlikte kullanılamaz^[4]
- Aynı `-k` deseni diğer komutlarda da geçerlidir: `kubectl get -k`, `describe -k`, `diff -k` (uygulanırsa oluşacak fark), `delete -k` ^[1]
- `apply --dry-run=client` nesneyi API'ye göndermeden yazdırır^[4]; cluster'sız ortamda tam inceleme `kubectl kustomize` ile yapılır

```
kubectl kustomize overlays/prod          # yalnız render, ağ yok
kubectl diff -k overlays/prod            # uygulanacak fark
kubectl apply -k overlays/prod --dry-run=client # nesne gönderilmez
```

Helm





Chart, Release ve Repository

- Helm, Kubernetes için bir paket yöneticisidir ("package manager"); dağıtım birimine chart denir^[6,9]

"A chart is a collection of files that describe a related set of Kubernetes resources."^[6]

"Chart, birbiriyle ilişkili Kubernetes kaynakları kümesini tanımlayan dosyalar koleksiyonudur."

- **Release:** cluster'a kurulmuş chart örneğidir; her install/upgrade/rollback revision numarasını 1 artırır, ilk revision her zaman 1'dir; `helm history` geçmiş listeler^[5]
- **Repository:** paketlenmiş chart'ları sunan HTTP sunucusudur; içindeki `index.yaml` paket listesini taşır^[6]
- Keşif: `helm search hub` Artifact Hub'ı tarar; `helm repo add` ile eklenen depolar `helm search repo` ile yerel olarak aranır^[5]



Chart Dizin Yapısı

```
mychart/  
  Chart.yaml      # zorunlu: apiVersion, name, version  
  values.yaml     # şablonlar için varsayılan değerler  
  templates/      # değerlerle birleşip manifest üreten şablonlar  
    NOTES.txt     # kurulum sonrası kullanıcıya basılan notlar  
  charts/         # bağımlı chart'lar (subchart'lar)  
  crds/           # şablonlanmayan CRD bildirimleri  
  .helmignore     # paketleme dışında kalacak dosyalar
```

- `Chart.yaml` zorunlu alanları: `apiVersion` (Helm 3 chart'larında `v2`), `name`, `version`; `kubeVersion` desteklenen Kubernetes sürüm kısıtı; `appVersion` paketlenen uygulamanın sürümü (bilgi amaçlı)^[6]
- `helm create AD` bu iskeleti hazır üretir; `helm package` chart'ı `AD-0.1.0.tgz` arşivine çevirir^[15,5]



Template ve Values

- `templates/` altındaki her dosya şablon motorundan geçer; motor Go template dili ve ek fonksiyon kütüphanesi (Sprig) kullanır^[6]
- Yönergeler `{{ }}` içinde yazılır: `{{ .Release.Name }}`, `{{ .Values.replicaCount }}` ^[8]
- `values.yaml` chart'ın varsayılanlarını taşır; kullanıcı `-f` veya `--set` ile override eder; kullanıcı değerleri chart varsayılanlarına göre önceliklidir^[6]
- Ön tanımlı nesneler her şablonda vardır ve override edilemez: `.Release` (Name, Namespace, IsInstall/IsUpgrade), `.Chart`, `.Capabilities` (KubeVersion, APIVersions), `.Files` ^[6]
- Zorunlu değerler `required` fonksiyonuyla işaretlenebilir; değer yoksa render hata verir^[6]

```
# templates/configmap.yaml
data:
  mesaj: {{ .Values.env.mesaj | quote }}
```



Values Önceliği

Öncelik düşükten yükseğe:^[7]

1. Chart'ın kendi `values.yaml` 'i
2. Subchart için üst chart'ın `values.yaml` 'i
3. `-f` ile verilen kullanıcı dosyası (birden çok `-f` verilirse en sağdaki kazanır)
4. `--set` parametreleri (en yüksek öncelik)
 - `--set` değerleri release kaydında saklanır; `helm get values` ile görüntülenir, `helm upgrade --reset-values` ile temizlenir^[5]
 - Bir anahtarı `null` yapmak, o anahtarı birleşmeden siler; varsayılandaki eski değer kalmaz^[7]

```
helm upgrade demo mychart -f prod-values.yaml --set image.tag=1.27.2
```



Release Yaşam Döngüsü

```
helm install demo mychart           # chart'tan release kurar
helm upgrade demo mychart -f prod.yaml # release'i yeni chart/değerlerle günceller
helm rollback demo 1                 # 1 numaralı revision'a döner
helm uninstall demo                  # kaynakları ve release kaydını siler
```

- `install` iki argüman alır: seçilen release adı ve chart; chart kaynağı repository referansı, yerel dizin, `.tgz` arşivi veya URL olabilir^[5]
- `upgrade` yalnız değişen kaynakları güncellemeye çalışır; `--reuse-values` önceki release'in değerlerini koruyup üstüne komut satırındakileri ekler^[5,10]
- `rollback` revision numarası ister; verilmezse bir önceki revision'a döner^[11]
- `uninstall` release kaydını da siler; silinen release'e geri dönülemez; `--keep-history` kaydı korur^[12,5]



Başarısızlık Koruması

- `--atomic` : install başarısız olursa kurulumu tamamen siler; upgrade başarısız olursa değişiklikleri geri alır; her ikisinde de `--wait` otomatik açılır^[9,10]
- `--wait` : Pod'lar hazır olana, PVC'ler bağlanana ve Deployment/StatefulSet/ReplicaSet en az Pod sayısına ulaşana kadar bekler; süre `--timeout` ile sınırlıdır^[9]
- `--timeout` varsayılanı 5 dakikadır^[9]
- `--dry-run` : kurulumu simüle eder; varsayılan `client` modu cluster bağlantısı kurmaz ve render edilen `MANIFEST` bölümünü gösterir^[9,8]

"It's best not to assume that your chart will install just because --dry-run works."^[8]
"dry-run çalıştı diye chart'ın gerçekten kurulacağını varsaymamak gerekir."



helm template ve helm lint: Yerel Doğrulama

- `helm template AD chart`: şablonları yerel olarak render edip yazar; cluster'dan okunacak değerler yerel olarak taklit edilir, sunucu tarafı geçerlilik denetimi yapılmaz^[13]
- `-s templates/x.yaml` tek şablonu gösterir; `--output-dir` çıktıyı dosyalara yazar; `--kube-version` ve `--api-versions` `.Capabilities` 'i taklit eder^[13]
- `helm lint chart`: kurulumu kıracak sorunlar için `[ERROR]`, sözleşme/öneri ihlalleri için `[WARNING]` üretir^[14]
- `--strict` uyarıları da hata sayar; `--with-subcharts` bağımlı chart'ları da denetler^[14]
- Cluster'a giden gerçek sonucu `helm get manifest <release>` ile okumak mümkündür; her belge hangi şablondan geldiğini söyleyen yorum satırıyla başlar^[8]

```
helm template demo mychart # tam render
helm template demo mychart -s templates/deployment.yaml
helm lint mychart --strict
```



Dependency'lere Giriş

- Bağımlılıklar `Chart.yaml` içindeki `dependencies` listesinde tanımlanır: `name`, `version` ve `repository` (veya `@takma-ad`)^[6]
- `helm dependency update` bağımlıları indirip `charts/` dizinine arşiv olarak koyar; `helm dependency build` bunu kilit dosyasından yeniden kurar^[16,6]
- Kurulumda üst chart ve tüm bağımlılar tek release altında birleşir; kaynaklar türe ve ada göre sıralanarak uygulanır^[6]
- Değerler isim uzayına bölünür: üst chart'ın `values.yaml` 'inde `mysql:` anahtarı subchart'a gider; subchart kendi şablonlarında `mysql.` önekini görmez^[6]
- `global:` bloğu tüm chart'lara yayılır; subchart'ların ortak ayarı için kullanılır^[6]

Kaynaklar





Kaynaklar

1. Declarative Management of Kubernetes Objects Using Kustomize: kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/
2. Declarative Management of Kubernetes Objects: kubernetes.io/docs/tasks/manage-kubernetes-objects/declarative-config/
3. kubectl kustomize: kubernetes.io/docs/reference/kubectl/generated/kubectl_kustomize/
4. kubectl apply: kubernetes.io/docs/reference/kubectl/generated/kubectl_apply/
5. Helm - Using Helm: helm.sh/docs/v3/intro/using_helm/
6. Helm - Charts: helm.sh/docs/v3/topics/charts/
7. Helm - Values Files: helm.sh/docs/v3/chart_template_guide/values_files/
8. Helm Chart Template Guide - Getting Started: helm.sh/docs/v3/chart_template_guide/getting_started/
9. helm install: helm.sh/docs/v3/helm/helm_install/
10. helm upgrade: helm.sh/docs/v3/helm/helm_upgrade/
11. helm rollback: helm.sh/docs/v3/helm/helm_rollback/
12. helm uninstall: helm.sh/docs/v3/helm/helm_uninstall/
13. helm template: helm.sh/docs/v3/helm/helm_template/
14. helm lint: helm.sh/docs/v3/helm/helm_lint/
15. helm create: helm.sh/docs/v3/helm/helm_create/
16. helm dependency: helm.sh/docs/v3/helm/helm_dependency/